

13 · Procedures, Functions and Packages

Why this matters in interviews. "Procedure vs function", "IN, OUT, IN OUT", "how do you call a procedure that has an OUT parameter?", "what is a package — spec vs body?" — stored code is the heart of any PL/SQL role.

In this chapter: subprograms and their parts · stored procedures · parameter modes and notations · functions (standalone, nested, recursive) · procedure vs function · packages (specification, body, public/private).

Assumed sample data for the examples (you can create it in seconds):

```
CREATE TABLE Employees (EmpID NUMBER PRIMARY KEY, Name VARCHAR2(50), Salary NUMBER, Department VARCHAR2(20));
INSERT INTO Employees VALUES (1, 'Alice', 50000, 'Sales');
INSERT INTO Employees VALUES (2, 'Bob', 60000, 'Sales');
INSERT INTO Employees VALUES (3, 'Charlie', 45000, 'HR');
COMMIT;
```

13.1 Subprograms

A **subprogram** is a named PL/SQL block that can take **parameters** and be **invoked**. There are two kinds:

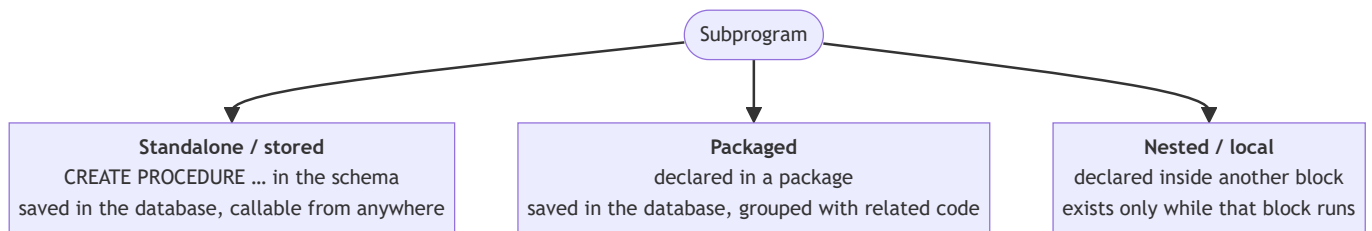
	Function	Procedure
Purpose	compute and return a single value	perform an action
Returns	one value, via RETURN	nothing directly (values go out through OUT parameters)
Called	inside an expression or SQL statement: x := f(3); SELECT f(a) FROM t	as a statement: p(3); or EXEC p(3);

Parts of a subprogram

Like an anonymous block, a subprogram has **three parts** (plus a name and an optional parameter list):

Part	Required?	Notes
Declarative part	optional	types, cursors, constants, variables, exceptions and nested subprograms. It does not start with the DECLARE keyword — it follows IS / AS . Items are local and cease to exist when the subprogram finishes
Executable part	mandatory	the statements that perform the action
Exception-handling part	optional	handles run-time errors

Where a subprogram can live



13.2 Procedures

Creating a procedure

```
CREATE [OR REPLACE] PROCEDURE procedure_name
    [(parameter_name [IN | OUT | IN OUT] type [, ...])]
{IS | AS}
BEGIN
    < procedure_body >
END procedure_name;
/
```

- **procedure_name** — the name.
- **[OR REPLACE]** — allows **modifying an existing** procedure without dropping it (privileges granted on it survive).
- The optional **parameter list** contains the parameter **name, mode and type**. **IN** is a value passed **from outside**; **OUT** is a parameter used to **return a value outside**.
- **procedure_body** contains the executable part.
- The **AS** keyword is used instead of **IS** for creating a **standalone** procedure (they are otherwise interchangeable).
- Parameter types are written **without a length**: `name IN VARCHAR2` , not `VARCHAR2(50)` .

```
CREATE OR REPLACE PROCEDURE greetings
AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Hello World!');
END;
/
```

Procedure created.

Executing a standalone procedure

Two ways: the **EXECUTE** keyword, or calling it **by name from a PL/SQL block**.

```
EXECUTE greetings;           -- also: EXEC greetings;
```

```
Hello World
```

```
PL/SQL procedure successfully completed.
```

```
BEGIN
    greetings;           -- called from another PL/SQL block
END;
/
```

`EXEC / EXECUTE` is a **SQL*Plus / SQL Developer client command**, not PL/SQL. Oracle also offers `CALL greetings();`.

Deleting a procedure

```
DROP PROCEDURE greetings;
```

Procedure with no parameters — a query loop

```
CREATE OR REPLACE PROCEDURE ShowEmployees IS
BEGIN
    FOR rec IN (SELECT * FROM Employees) LOOP
        DBMS_OUTPUT.PUT_LINE('ID: ' || rec.EmpID || ', Name: ' || rec.Name);
    END LOOP;
END ShowEmployees;
/
```

To run it and see the output:

```
SET SERVEROUTPUT ON;
BEGIN
    ShowEmployees;
END;
/
```

```
ID: 1, Name: Alice
ID: 2, Name: Bob
ID: 3, Name: Charlie
```

`rec` is an **implicit record variable**: each row returned by the query in parentheses becomes a record whose **fields are the selected columns**, accessed as `rec.column_name` (Chapter 14).

Procedure with an **IN** parameter and an exception handler

```
CREATE OR REPLACE PROCEDURE GetEmployeeByID (emp_no IN NUMBER) IS
    v_name VARCHAR2(50);
BEGIN
    SELECT Name INTO v_name FROM Employees WHERE EmpID = emp_no;
    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_name);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee found with ID ' || emp_no);
END GetEmployeeByID;
/
```

```
BEGIN
    GetEmployeeByID(2);
END;
/
```

Employee Name: Bob

GetEmployeeByID(99) prints No employee found with ID 99 — SELECT ... INTO found no row, raising NO_DATA_FOUND, which the EXCEPTION section caught.

Procedure with an **OUT** parameter

```
CREATE OR REPLACE PROCEDURE GetTotalSalary (total OUT NUMBER) IS
BEGIN
    SELECT SUM(Salary) INTO total FROM Employees;
END GetTotalSalary;
/
```

```
DECLARE
    total_sal NUMBER;
BEGIN
    GetTotalSalary(total_sal);
    DBMS_OUTPUT.PUT_LINE('Total Salary: ' || total_sal);
END;
/
```

Total Salary: 155000

Procedure with IN and OUT

```
CREATE OR REPLACE PROCEDURE GetDeptCount (  
    dept      IN  VARCHAR2,  
    emp_count OUT NUMBER  
) IS  
BEGIN  
    SELECT COUNT(*) INTO emp_count  
    FROM   Employees  
    WHERE  Department = dept;  
END GetDeptCount;  
/
```

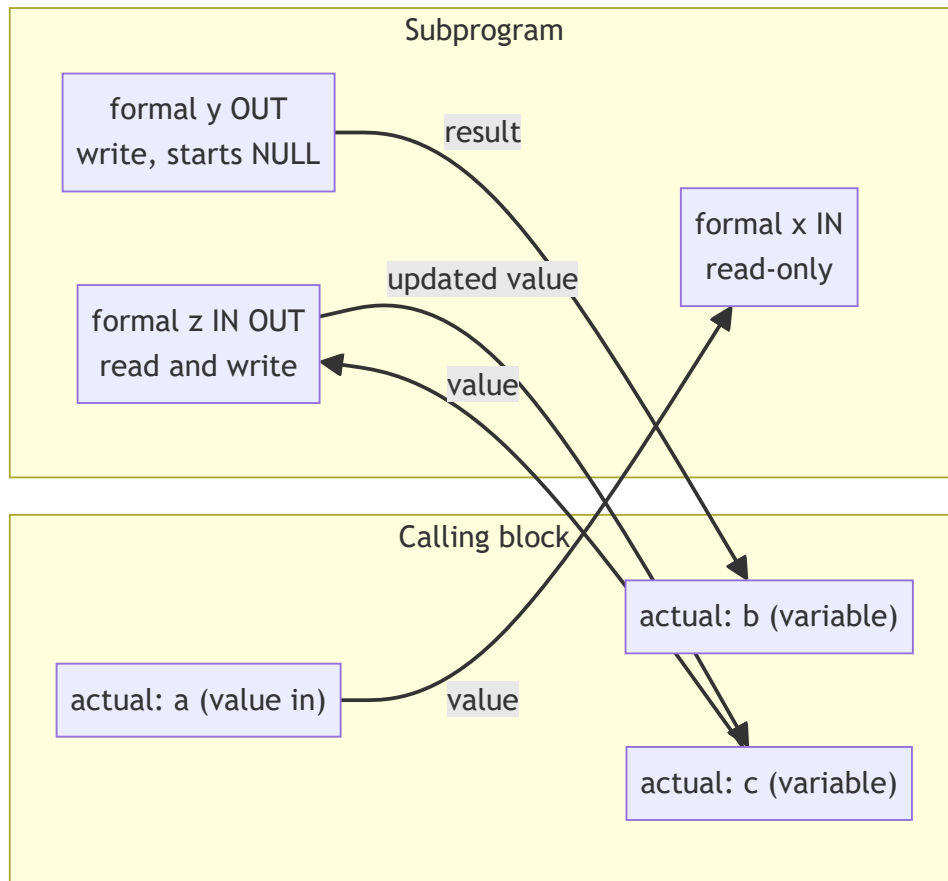
```
DECLARE  
    count_emp NUMBER;  
BEGIN  
    GetDeptCount('Sales', count_emp);  
    DBMS_OUTPUT.PUT_LINE('Employees in Sales: ' || count_emp);  
END;  
/
```

Employees in Sales: 2

13.3 Parameter modes

Parameters are how data is passed **into or out of** a subprogram. There are **three modes**:

Mode	Direction	Description
IN	input only	passes a value into the subprogram
OUT	output only	returns a value from the subprogram
IN OUT	input and output	passes a value in, and modifies it



IN — the default

- **Default mode** (write **IN** or leave it out).
- Used to **send a value** to the procedure. It is a **read-only parameter**: inside the subprogram it acts **like a constant** and **cannot be changed**.
- You may pass a **constant, literal, initialised variable or expression**.
- It can have a **default value**; in that case it may be **omitted from the call**.
- Passed **by reference** (no copy is made).

```

CREATE OR REPLACE PROCEDURE ShowSquare (num IN NUMBER) IS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Square: ' || (num * num));
END;
/

BEGIN
    ShowSquare(4);
END;
/

```

Square: 16

OUT — return a value

- Used to **return a value** from the procedure — it acts like a **result holder**.

- Inside the subprogram an **OUT** parameter behaves like a **variable**: you can change and read it after assigning. On entry it is **not initialised** (NULL) — it **cannot carry a value in**.
- The **actual parameter must be a variable** (declared *before* the call), never a literal or expression.
- Passed **by value** (copied out when the subprogram ends normally).

```
CREATE OR REPLACE PROCEDURE GetPi (pi OUT NUMBER) IS
BEGIN
    pi := 3.14159;
END;
/

DECLARE
    my_pi NUMBER;
BEGIN
    GetPi(my_pi);
    DBMS_OUTPUT.PUT_LINE('Value of Pi: ' || my_pi);
END;
/
```

Value of Pi: 3.14159

IN OUT — read and update

- Accepts a value **into** the procedure, which may be **modified and returned out**.
- Useful when you want to **read and update** the same value.
- The actual parameter must be a **variable** (not a constant or expression) and must hold a value; passed **by value**.

```
CREATE OR REPLACE PROCEDURE AddTen (num IN OUT NUMBER) IS
BEGIN
    num := num + 10;
END;
/

DECLARE
    x NUMBER := 25;
BEGIN
    AddTen(x);
    DBMS_OUTPUT.PUT_LINE('Updated value: ' || x);
END;
/
```

Updated value: 35

The same idea in the lab, with the reminder "use **IN OUT** because an **OUT** parameter cannot carry the value inside":

```

CREATE OR REPLACE PROCEDURE p (
    x IN OUT NUMBER          -- use IN OUT because an OUT parameter cannot carry the value inside
) IS
BEGIN
    x := x + 10;
END;
/

SET SERVEROUTPUT ON;
DECLARE
    n NUMBER;
BEGIN
    n := 10;
    p(n);
    DBMS_OUTPUT.PUT_LINE(n);
END;
/

```

20

IN and OUT together — a *local* procedure

A procedure can also be declared **inside a block's declaration section** (a *local* procedure), defined and called in the same program. This one takes two **IN** numbers and returns their minimum through an **OUT** :

```

DECLARE
    a NUMBER;
    b NUMBER;
    c NUMBER;
    PROCEDURE findMin(x IN NUMBER, y IN NUMBER, z OUT NUMBER) IS
    BEGIN
        IF x < y THEN
            z := x;
        ELSE
            z := y;
        END IF;
    END;
BEGIN
    a := 23;
    b := 45;
    findMin(a, b, c);
    DBMS_OUTPUT.PUT_LINE(' Minimum of (23, 45) : ' || c);
END;
/

```

Minimum of (23, 45) : 23

And an **IN OUT** example — the square of a passed value, computed **in place**:

```

DECLARE
  a NUMBER;
  PROCEDURE squareNum(x IN OUT NUMBER) IS
  BEGIN
    x := x * x;
  END;
BEGIN
  a := 23;
  squareNum(a);
  DBMS_OUTPUT.PUT_LINE(' Square of (23): ' || a);
END;
/

```

Square of (23): 529

Formal vs actual parameters

	Where	Example	Meaning
Formal	in the procedure definition	x, y, z, m in findMin(x IN ..., y IN ..., z OUT ..., m OUT ...)	placeholders for incoming/outgoing values
Actual	in the call	a, b, c, d in findMin(a, b, c, d)	the real variables/values passed

Notations for passing parameters

Assume findMin(x, y, z, m):

Notation	Call	Rule
Positional	findMin(a, b, c, d);	1st actual → 1st formal, 2nd → 2nd, ... (so a→x, b→y, c→z, d→m)
Named	findMin(x => a, y => b, z => c, m => d);	each actual is tied to a formal name with the arrow =>; any order
Mixed	findMin(a, b, c, m => d);	✅ legal — positional first, then named
Mixed, wrong way	findMin(x => a, b, c, d);	❌ not legal — a positional argument cannot follow a named one

Named notation is self-documenting and lets you **skip parameters that have defaults**:

```

CREATE OR REPLACE PROCEDURE raise_salary (p_id IN NUMBER, p_pct IN NUMBER DEFAULT 10) IS
BEGIN
    UPDATE Employees SET Salary = Salary * (1 + p_pct / 100) WHERE EmpID = p_id;
END;
/

BEGIN
    raise_salary(1);                -- uses the default 10 %
    raise_salary(p_id => 2, p_pct => 5); -- named
END;
/

```

How to call a procedure that has OUT / IN OUT parameters

EXEC works directly when the procedure has **no parameters or only IN parameters**:

```

EXEC HelloWorld;                -- no parameters
EXECUTE HelloWorld;
EXEC Greet('Shrishaanth');      -- an IN parameter

```

For OUT / IN OUT, the call needs **somewhere to put the result**. There are two correct ways:

1 · An anonymous block (the usual way) with a declared variable:

```

DECLARE
    x NUMBER;
BEGIN
    GetPi(x);
    DBMS_OUTPUT.PUT_LINE('Pi: ' || x);
END;
/

```

2 · A SQL*Plus / SQL Developer bind variable — VARIABLE declares it, :x refers to it, PRINT shows it:

```

VARIABLE x NUMBER
EXEC GetPi(:x)
PRINT x

```

```

          X
-----
    3.14159

```

Correction. A common claim is that "EXEC cannot be used for OUT parameters". It **can** — but only with a **bind variable** (:x declared with VARIABLE). Writing EXEC GetPi(:x); *without* first declaring x fails with SP2-0552: Bind variable "X" not declared. Complex multi-statement logic still belongs in a BEGIN ... END; block.

13.4 Functions

A **function** is like a procedure except that it **must return a single value** and is normally used **inside an expression**.

Creating a function

```
CREATE [OR REPLACE] FUNCTION function_name
    [(parameter_name [IN | OUT | IN OUT] type [, ...])]
RETURN return_datatype
{IS | AS}
BEGIN
    < function_body >
END [function_name];
/
```

- The function **must contain a RETURN statement**.
- The **RETURN clause** in the header specifies the **data type** of the value returned.
- `[OR REPLACE]` modifies an existing function; `AS` is used instead of `IS` for a standalone function.

Standalone vs nested functions

A **standalone function** is **created independently in the database** — *not* inside a package, procedure or block. It is stored as a **separate schema object**, just like a table or view, so it can be used by other procedures, functions, triggers and (with restrictions) SQL statements. It exists **until dropped** with `DROP FUNCTION`.

```
CREATE OR REPLACE FUNCTION get_square (n NUMBER)
RETURN NUMBER
AS
    -- IS also works
BEGIN
    RETURN n * n;
END;
/
```

```
SELECT get_square(5) FROM dual;      -- 25
```

A **non-standalone (nested or packaged) function** exists **only inside its block** (or package): it is **not saved** in the database as a schema object and disappears when the block finishes.

```
DECLARE
    FUNCTION get_square (n NUMBER) RETURN NUMBER IS
    BEGIN
        RETURN n * n;
    END;
BEGIN
    DBMS_OUTPUT.PUT_LINE(get_square(5));
END;
/
```

Example — maximum of two numbers

```

DECLARE
  a NUMBER;
  b NUMBER;
  c NUMBER;
  FUNCTION findMax(x IN NUMBER, y IN NUMBER) RETURN NUMBER IS
    z NUMBER;
  BEGIN
    IF x > y THEN
      z := x;
    ELSE
      z := y;
    END IF;
    RETURN z;
  END;
BEGIN
  a := 23;
  b := 45;
  c := findMax(a, b);
  DBMS_OUTPUT.PUT_LINE(' Maximum of (23, 45): ' || c);
END;
/

```

Maximum of (23, 45): 45

Recursive functions

When a subprogram **calls itself** the call is **recursive** and the process is **recursion**. Factorial: $n! = n \times (n-1)! = n \times (n-1) \times (n-2)! \dots \times 1$, with $0! = 1$ as the **base case** that stops the recursion.

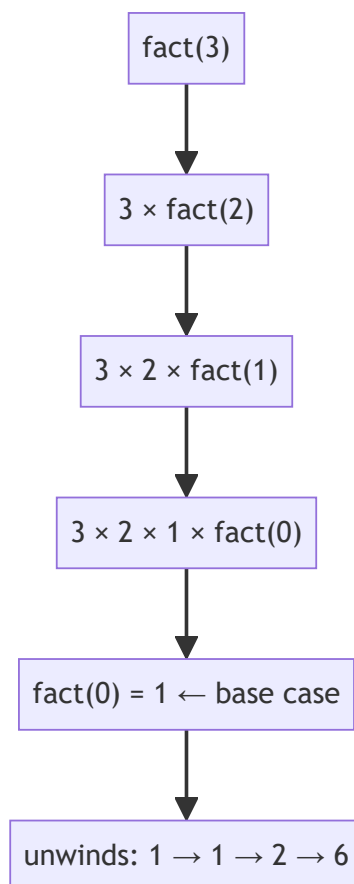
```

DECLARE
    num      NUMBER;
    factorial NUMBER;

    FUNCTION fact(x NUMBER) RETURN NUMBER IS
        f NUMBER;
    BEGIN
        IF x = 0 THEN
            f := 1;           -- base case
        ELSE
            f := x * fact(x - 1); -- recursive call
        END IF;
        RETURN f;
    END;
BEGIN
    num      := 6;
    factorial := fact(num);
    DBMS_OUTPUT.PUT_LINE(' Factorial ' || num || ' is ' || factorial);
END;
/

```

Factorial 6 is 720



Using functions in SQL

```
SELECT emp_name, get_square(salary / 1000) AS score FROM emp;
SELECT * FROM emp WHERE get_square(dept_id) > 100;
```

A function called **from a SELECT** must obey restrictions: it **cannot perform DML** (INSERT / UPDATE / DELETE), **cannot COMMIT / ROLLBACK**, all its parameters must be **IN**, and it must return an **SQL data type** — in particular **not BOOLEAN**. (Called from PL/SQL these limits do not apply.)

Things that go wrong

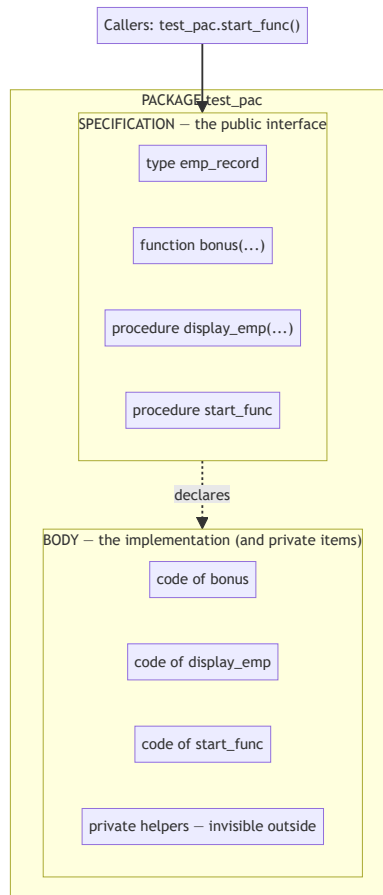
Situation	Result
A function reaches <code>END</code> without executing <code>RETURN</code>	<code>ORA-06503: PL/SQL: Function returned without value</code>
<code>RETURN</code> in a procedure	allowed — it just exits the procedure early (no value)
Calling a function like a procedure: <code>get_square(5);</code>	<code>PLS-00221: 'GET_SQUARE' is not a procedure or is undefined</code> — a function call must be part of an expression
Recursion with no base case	runs until the session exhausts its memory (e.g. <code>ORA-04030</code>) — always have a base case

Procedure vs function — the summary

	Procedure	Function
Returns a value	not directly (via <code>OUT</code>)	must return exactly one (<code>RETURN</code>)
<code>RETURN</code> clause	none (a bare <code>RETURN;</code> exits)	mandatory (with data type)
Called as	a statement	part of an expression / SQL
Usable in <code>SELECT</code>	✗	✓ (with restrictions)
Typical use	do something: insert, update, report	compute something: tax, discount, formatted value
<code>OUT</code> parameters	common	possible but discouraged

13.5 Packages

A **package** is a **schema object that groups logically related PL/SQL types, variables, constants, cursors, procedures and functions**. It has **two parts**, each stored separately:



Package specification

The **specification** is the **interface** to the package. It **only declares** the types, variables, constants, exceptions, cursors and subprograms that can be **referenced from outside** — everything a caller may use, but **not the code** of the subprograms.

- All objects placed in the specification are called **public** objects.
- Any subprogram **not** in the specification but coded in the **body** is a **private** object.
- You can have many global variables and multiple procedures or functions in one package.

```
CREATE PACKAGE cust_sal AS
    PROCEDURE find_sal(c_id customers.id%TYPE);
END cust_sal;
/
```

Package created.

Package body

The **body** holds the **code of the methods declared in the specification**, plus any **private declarations hidden from outside** the package.

```
CREATE OR REPLACE PACKAGE BODY cust_sal AS

    PROCEDURE find_sal(c_id customers.id%TYPE) IS
        c_sal customers.salary%TYPE;
    BEGIN
        SELECT salary INTO c_sal
        FROM    customers
        WHERE   id = c_id;
        DBMS_OUTPUT.PUT_LINE('Salary: ' || c_sal);
    END find_sal;

END cust_sal;
/
```

Package body created.

```
EXECUTE cust_sal.find_sal(1);           -- Salary: 2000    (customer 1 = Ramesh)
```

Callers use `package_name.item_name` .

A fuller example — record, function and two procedures

```
CREATE OR REPLACE PACKAGE test_pac IS
    TYPE emp_record IS RECORD (
        id      NUMBER,
        name    VARCHAR2(50),
        salary  NUMBER
    );
    FUNCTION bonus(salary NUMBER) RETURN NUMBER;
    PROCEDURE display_emp(e IN emp_record);
    PROCEDURE start_func;
END test_pac;
/

CREATE OR REPLACE PACKAGE BODY test_pac IS

    FUNCTION bonus(salary IN NUMBER) RETURN NUMBER IS
        s NUMBER;
    BEGIN
        s := salary * 0.1;
        RETURN s;
    END bonus;

    PROCEDURE display_emp(e IN emp_record) IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Id : ' || e.id);
        DBMS_OUTPUT.PUT_LINE('Name : ' || e.name);
        DBMS_OUTPUT.PUT_LINE('Salary : ' || e.salary);
    END display_emp;

    PROCEDURE start_func IS
        res NUMBER;
        r1 emp_record;
    BEGIN
        r1.id      := 1;
        r1.name    := 'Arjun';
        r1.salary  := 10000;
        res := bonus(r1.salary);
        DBMS_OUTPUT.PUT_LINE('Bonus : ' || res);
        display_emp(r1);
    END start_func;

END test_pac;
/

SET SERVEROUTPUT ON;
BEGIN
    test_pac.start_func();
```

```
END;  
/
```

```
Bonus : 1000  
Id : 1  
Name : Arjun  
Salary : 10000
```

Notes on this example:

- The **record type** `emp_record` lives in the **specification**, so callers (and other packages) can declare variables of that type.
- The declaration `bonus(salary NUMBER)` in the spec and `bonus(salary IN NUMBER)` in the body **must match** — `IN` is the default, so they do.
- `start_func` is the only entry point a caller needs; it calls `bonus` and `display_emp` **without** the package prefix because they are in the same package.
- A subprogram that appears **only in the body** (say a helper `log_it`) would be **private**: callable from inside the package, invisible outside.

Why packages?

Benefit	Explanation
Encapsulation	public interface vs hidden implementation
Modularity	related code grouped; one name to remember
Performance	the whole package is loaded into memory on first use — later calls avoid disk I/O
Easier maintenance	changing a body does not invalidate programs that depend only on the specification
Persistent state	package-level variables keep their values for the whole session
Overloading	several subprograms with the same name and different parameters

Working with them

```
DROP PACKAGE test_pac;           -- spec + body  
DROP PACKAGE BODY test_pac;      -- body only  
SELECT object_name, object_type, status FROM user_objects WHERE object_type LIKE 'PACKAGE%';  
SELECT text FROM user_source WHERE name = 'TEST_PAC' ORDER BY type, line;
```

13.6 When a compile goes wrong

A subprogram created with errors is stored but marked **INVALID** — you get *"Warning: Procedure created with compilation errors."*

```
SHOW ERRORS -- errors for the last object you compiled
SHOW ERRORS PROCEDURE greetings
SELECT line, position, text FROM user_errors WHERE name = 'GREETINGS';
SELECT object_name, status FROM user_objects WHERE status = 'INVALID';
```

Error	Usual cause
PLS-00201: identifier 'X' must be declared	typo, missing declaration or missing privilege
PLS-00103: Encountered the symbol ...	syntax error — often a missing <code>;</code> or <code>END</code>
PLS-00306: wrong number or types of arguments	call doesn't match the formal parameters
PLS-00363: expression cannot be used as an assignment target	assigning to a constant or an <code>IN</code> parameter
ORA-04063: ... has errors	calling a subprogram that is <code>INVALID</code>

Interview corner

Q. Procedure vs function? A function must return exactly one value and is used inside expressions/SQL; a procedure performs an action and returns values only through `OUT` parameters.

Q. IN, OUT, IN OUT? `IN` — read-only input (default, by reference); `OUT` — output only, starts `NULL`, actual must be a variable; `IN OUT` — passes a value in and returns the modified value.

Q. How do you call a procedure with an OUT parameter? From an anonymous block with a declared variable, or in SQL*Plus/SQL Developer with `VARIABLE x ...; EXEC p(:x); PRINT x;`

Q. Can you use a procedure in a SELECT ? A function? No / yes (a function, if it doesn't do DML, doesn't commit and returns an SQL type).

Q. What is the difference between positional and named notation? Positional matches by order; named matches by formal name using `=>`. Mixed is legal only with positional arguments first.

Q. What is a package? Specification vs body? A schema object grouping related types, variables, procedures and functions. The spec declares the **public** interface; the body implements it and may hold **private** items.

Q. Advantages of packages? Encapsulation, modularity, better performance (loaded once), session-level state, overloading, and dependency isolation (body changes don't invalidate dependents).

Q. What does CREATE OR REPLACE do? Replaces the definition if the object exists (keeping grants), creates it otherwise.

Q. What is recursion in PL/SQL? What must you include? A subprogram calling itself; always a base case.

Q. Standalone vs local subprogram? Standalone is a schema object stored in the database (callable anywhere); a local (nested) one is declared inside another block and exists only while that block runs.

Q. Where does the declarative part of a subprogram start? After `IS / AS` — with no `DECLARE` keyword.

Quick recap

- **Function** = returns one value (`RETURN`), used in expressions/SQL. **Procedure** = performs an action, values leave via `OUT`.

- Header: `CREATE [OR REPLACE] PROCEDURE|FUNCTION name (p IN|OUT|IN OUT type) [RETURN type] IS|AS ... BEGIN ... END;` — declarations follow `IS / AS`, **no DECLARE**; parameter types have **no length**.
- Run a procedure with `EXEC name(...)` (client command) or from a block; `OUT / IN OUT` **need a variable** (block variable or `VARIABLE / :x` bind).
- `IN` read-only (by reference) · `OUT` write-only starting NULL (by value) · `IN OUT` both. Notation: **positional**, **named** (`=>`), **mixed** (positional first).
- **Standalone** (stored), **packaged**, **nested/local**. Recursion needs a base case.
- **Package** = **specification** (public interface) + **body** (implementation + private items); call as `pkg.item`; benefits: encapsulation, performance, state, overloading.